

Herat Heritage Watch — student challenge

hackathon/README.md · branch main · commit 8acc781 · rendered 2026-09-23
<https://github.com/whamidou006/heritage-watch-release>

Classify **what kind of change** happened at a location in Herat Old City, Afghanistan, from **two aerial photographs of the same place, taken years apart**.

You are given the location. You do not have to find the change — you have to say what it was.

Class	Labels	What it looks like
New Construction	382	bare ground or rubble → a standing building
Solar Panel	309	a rooftop or ground-mounted panel array appears
Destruction	138	a standing building → rubble or bare ground
Temporary Structure	53	tents, tarpaulins, informal shelter
Total	882	

882 labelled points across 10 acquisition intervals between 2009 and 2025.

This is *semantic* change detection: one-to-many, not "did something change". Searching for "change detection" will get you the wrong papers.

Run the baseline in three steps

```
git clone https://github.com/whamidou006/heritage-watch-release
cd heritage-watch-release
pip install -e . # installs the evaluation harness

unzip dataset.zip -d raw # supplied by the project partner

cd hackathon
python baseline.py
```

That is the whole path. The first run cuts 64 px chips (~25 s, 16 MB) and then prints both official scores. CPU only, about two minutes. Each score runs 8 replicates of a full cross-validation and prints only when a block finishes, so a quiet minute is normal.

The data

The imagery is distributed by the project partner, not from this repository. You will be given `dataset.zip` (306 MB). Unzip it beside the repo:

```
unzip dataset.zip -d raw
```

The code in this repository is MIT-licensed. The imagery is not — use it for the challenge only, and do not redistribute it.

```

raw/dataset/
├── herat_site_sat_images/herat_site_TM_z19/*.tif    21 aerial scenes, 2009–2025
├── Herat_all_changes.csv                          the analyst's labels
└── shortlist_sites.geojson                       site polygons

```

What the imagery is. 21 true-colour GeoTIFF scenes of one 2.2 km square of Herat Old City at ~0.25 m/px, spanning 2009 to 2025. Every scene sits on the same CRS, bounds and pixel grid, so a before-chip and an after-chip at the same point are **pixel-aligned by construction** — you never co-register anything. No scene has blank or nodata pixels inside the footprint.

What the labels are. An analyst marked points where something changed and typed each one. `data/manifest.json` (shipped here) is the cleaned version: 884 points with class, coordinates, the acquisition pair, and the two scene filenames. It is the same file the reference numbers were measured from.

Two traps in the raw CSV, already fixed in `manifest.json`: the `x` column header has a **trailing space**, and coordinates use **comma decimal separators**. A naive `pandas.read_csv` corrupts both silently.

How 1,370 annotations became 882:

Step	Remaining
Raw annotations	1,370
– no scene within ± 75 days of the stated month	1,260
– point outside the imagery footprint	910
– classes <code>Other</code> and <code>Reconstruction</code>	884
– chip window clipped at the raster edge (at 64 px)	882

Only the last step depends on chip size: 64 px drops 2 points, 128 px drops 5. **Always join on `fid`, never on row order.**

The chips

`baseline.py` cuts them for you. To cut them yourself, or at another size:

```

cd hackathon
python make_chips.py --images ../raw/dataset --chip 64 --out chips_64.npz

```

You get `t1`, `t2` as `(N, 64, 64, 3) uint8`, plus `y`, `fid`, `pair`, `x`, `yy`.

You get **pixels, not embeddings**, on purpose: shipping features would steer everyone toward the two encoders we happened to try, and the most promising directions here need the raw imagery anyway.

`--chip` is a real hyperparameter. 64 is the default because a paired sweep put it ahead of 128 by 0.0885 macro-F1 on 8 of 8 replicates. What matters is the **ground extent**, not the pixel count: 64 px spans about 16 m, roughly one building.

The evaluation protocol

Why there are two scores

Knowing nothing but **when** a pair was taken scores **0.405** — nearly three times the 0.151 floor — because different kinds of change happened in different years, and the classes are not spread evenly across the ten intervals. A model given imagery can quietly bank that 0.405 without ever looking at a building.

Held out from its own acquisition, date-only falls to **0.174**, just above its 0.111 floor. That collapse is the entire reason this benchmark has two numbers. `baseline.py` runs this demonstration every time, so you can watch it happen.

The two scores

You report **both**.

Score	Question	How it splits
<code>evaluate</code>	Another building, same flight	Test points are whole spatial cells; every acquisition appears on both sides
<code>evaluate_interval</code>	An acquisition you have never seen	Each acquisition is held out in turn; its test cells are withheld from training too

The gap between them is large — on the colour baseline, 0.641 against 0.350, and every system we have run shows the same drop. A single number hides it.

Both return **macro-F1** over the four classes — every class counts equally, so you cannot win by getting the two big ones right. Both average 8 replicates with a different random grid origin each time and report the spread.

```
from heritage_watch.protocol import load_chips, evaluate, evaluate_interval, report

t1, t2, y, meta = load_chips("chips_64.npz")
X = my_model(t1, t2)           # frozen embeddings, a fine-tuned net, anything

report(evaluate(X, y, meta))
report(evaluate_interval(X, y, meta))
```

Or pass your own estimator and keep the features raw:

```
evaluate(X, y, meta, clf_factory=lambda: MyClassifier())
```

The rule for "better"

A difference counts as real only if it clears **all three** bars:

1. **Consistent** — wins on all 8 replicates.
2. **Bigger than the spread** — exceeds $2 \times$ the standard error of the paired per-replicate differences.
3. **At least 0.02 macro-F1** — the smallest gap this data can resolve.

```
from heritage_watch.protocol import compare
compare(mine, baseline, "mine", "baseline")    # applies all three bars
```

Clearing bars 1 and 2 but not 3 means "consistent but too small to matter". That is a normal, publishable outcome, and so is "within noise". Comparisons must be **paired** — same rows, same grids, same folds; `compare` checks this and refuses arms that do not match. Scoring two systems on different subsets and subtracting is the easiest way to invent a result that is not there.

Rules

1. **Change anything inside the model.** Swap the encoder, fine-tune it, replace the head, change the chip size, engineer features, ensemble.
2. **Every number you report comes from** `evaluate` / `evaluate_interval`. Labels cluster by construction site, so plain random cross-validation puts neighbouring pixels in train and test and inflates macro-F1. Do not write your own CV.
3. **Fit everything inside the fold** — scalers, PCA, feature selection, class weights.
4. **Any supervised training happens inside the fold.** Fine-tuning is encouraged, but do it in `clf_factory` on the training indices only. If you fine-tune a backbone on all 882 labels and then pass the embeddings in, you have leaked the test labels into every fold and the score is meaningless.
5. **Do not change the evaluation** — not the folds, the blocking, the replicate count, the class set, or the metric. Report a different protocol *next to* these two numbers, never instead of them.
6. **Final scoring uses a seed you do not have.** Development runs on a fixed public seed so every team sees identical partitions; the organisers re-score with an unpublished jitter seed. This removes *partition* overfitting only — all 882 labels are public, so declare your final system before asking for it to be scored.

Where you stand

Everything below is measured on 64 px chips, $n = 882$, 8 replicates, and every row is reproduced by `baseline.py` on CPU in about two minutes.

	another building	unseen acquisition
Majority-class floor	0.151	0.111
Date only (no imagery at all)	0.405	0.174
Colour histogram, after-image only	0.624	0.406
Colour histogram, both dates + difference	0.641	0.350

Beat 0.641 and 0.350, and say which score you moved.

Look at the last two rows. On score 1 adding the before-image gains +0.0167 on 8 of 8 replicates — consistent, but under 0.02, so the protocol calls it *not separated*. On score 2 the same change **loses** 0.0566 on 8 of 8, which is resolved. A modelling choice can help on one score and hurt on the other; that is exactly why you report both.

This is a deliberately weak starting point: 144 numbers per chip, no shape, no texture, no pretrained weights. A frozen off-the-shelf encoder and the same linear head will beat it, and that is the intended first move.

We are not publishing our own leaderboard entry. You are being scored against the protocol, not against us, and a target number tends to become a ceiling — people stop when they reach it. What we will say is

that the organisers' own systems are all **frozen and untuned**, so nothing you see here is near a ceiling, and that **Destruction is the weak class in every system we ran**: around 62 % correct, with roughly 29 % of it read as New Construction — the same two ground states in the opposite temporal order.

Things already tested, so you don't repeat them

Directions, not numbers — these were measured on systems stronger than the baseline, so the sizes would not carry over anyway.

- **Class reweighting is worth about -0.001 .** Balancing the head does nothing.
- **More labels alone will not fix Destruction.** Downsample New Construction to Destruction's support and the ranking flips, so Destruction is not intrinsically hard — it is *confusable* with one specific other class.
- **Chip size follows an inverted U** across 32 / 64 / 128 / 256 px, peaking at 64. The 64-versus-128 gap is 0.0885, 8/8, resolved, and it is the largest single lever anyone here has measured.
- **Adding the date as an input is a null** (+0.0010), which bounds the date shortcut but does not remove it — a model can use a correlate of the date without being handed the date.
- **The before-image may not be helping you.** Under score 1 every bi-temporal-versus-post-only contrast we ran came out too small to resolve. Under score 2 the colour baseline is *worse* with the before-image than without it (-0.0566 , 8/8, resolved). Using both dates well is an open problem, not a given.

Ideas worth trying

- **Any pretrained encoder:** DINOv2/v3, SatlasPretrain, Clay, a plain ImageNet CNN. Frozen embeddings plus a linear head is cheap, strong, and the obvious first move past the colour baseline.
- **Fine-tune a backbone instead of freezing it.** Nobody here has tried it — every system we have measured uses frozen features and a linear head.
- **Purpose-built semantic change detection:** Open-CD, SCanNet, ChangeMask, AnyChange (SAM-based proposals), ChangeStar / Changen2 for synthetic pairs.
- **Attack Destruction vs New Construction.** That single confusion is most of the remaining error, and it is a question of *direction*, not appearance.
- **Close the acquisition gap.** Most of the drop from score 1 to score 2 is genuine date novelty rather than a smaller training set, and colour or seasonal normalisation across dates is untried.
- **Multi-scale chips.** The sweep that chose 64 px only ever showed one window per point.

Submitting

Report, for **both** scores: macro-F1, the spread across replicates, and per-class F1. Include the `compare` verdict against the baseline and say what you changed. A submission that loses but reports honestly is more useful than one that wins by scoring a different split.

The protocol is specified in full in [docs/PROTOCOL.md](#) — the splitting, the decision rule, and the calibration behind the three bars. Read it if you want to audit the evaluation rather than just run it.

The organisers' own measurements are in `docs/RESULTS.md` and in the long report on the `research` branch. Nothing stops you reading them, but they are not the target: you are scored against the protocol, and a number to match is a worse goal than a question to answer.

A PDF of this page is at <docs/pdf/student-challenge.pdf>. It is rendered from this file by `scripts/make_pdf.py`; if the two ever disagree, this file is right.